Entwurfsdokument

HALLO FRISCH

Gruppe 9

Inhaltsverzeichnis

1. Einleitung **3**

2. Systementwurf **4**

2.1 Entwurfsziele 4

2.2 Systemzerlegung 8

2.3 Verwendung existierender Software-Komponenten 9

2.4 Management persistenter Daten 13

3. Objektentwurf **14**

3.1 Abwägung des Objektentwurfs 14

3.2 Klassenmodell der Entitäten 15

3.3 Dokumentation weiterer interessanter Ausschnitte des Entwurfsklassenmodells 16

4. Anhang **20**

1. Einleitung

Die zu implementierende Anwendung ist ein Lebensmittelbestellsystem. Das System stellt je nach Benutzer verschiedene Optionen bereit. Besagte Benutzer sind entweder Kunde, Geschäftsführer, Lieferant oder Einkäufer. Über ein gemeinsames Interface kann sich der Nutzer einloggen oder ein neues Kundenkonto erstellen bzw. sich als Kunde registrieren.

Beim Einloggen mit eindeutigem Nutzername und zugehörigem Passwort wird der Nutzer je nach Rolle zu seiner Benutzeroberfläche weitergeleitet.

Jedem Kunden wird ein eindeutiger Warenkorb zugeordnet. In diesen kann er Produkte in gewünschter Menge hinzufügen um anschließend eine Bestellung aufgeben. Weitere Funktionen bilden das Einsehen seiner vorangegangenen Bestellungen, sowie das Bearbeiten seiner noch offenen Bestellungen. Außerdem kann ein Kunde sein Profil bearbeiten. Für das Navigieren zu den gewünschten Funktionen steht dem Kunde eine Navigationsleiste bereit. Diese ermöglicht es dem Kunden außerdem nach spezifischen Produktwünschen zu filtern oder Produkte direkt zu suchen.

Produktdaten sowie Bestelldaten werden hierbei persistent vom System verwaltet. Auf sie kann von verschiedenen Akteuren zugegriffen werden. Je nach Akteur unterscheiden sich die diesbezüglich bereitgestellten Funktionen.

Ein Geschäftsführer kann anders als ein Kunde keine Bestellungen tätigen, sondern bekommt ein Interface mit administrativen Funktionen bereitgestellt. Er kann verschiedene Statistiken einsehen oder unter anderem das Sortiment verwalten, indem er neue Produkte anlegt, bereits vorhandene Produkte anpasst oder löscht. Weiterhin kann er neue Nutzerkonten für Mitarbeiter anlegen (Geschäftsführer, Lieferanten und Einkäufer).

Einkäufer können die Bestände der Produkte einsehen und extern getätigte Nachbestellungen eintragen und nach Lieferung einpflegen.

Lieferanten können Kundenbestellungen annehmen und ausliefern. Hierbei wird der, für den Lieferanten und Kunden einsichtbare, Status geändert.

In folgenden Abschnitten des Entwurfsdokuments werden der Systementwurf, sowie die Umsetzung einzelner Systemfunktionen spezifischer erläutert, um dem Leser einen besseren Einblick in die Funktionsweise der Software zu bieten.

2. Systementwurf

2.1 Entwurfsziele

Grundsätzlich sollte bei allen Kriterien der Kunde im Vordergrund stehen, damit eine nahtlose Einkaufserfahrung garantiert werden kann. Nützlichkeit und Nutzbarkeit sind bei allen Abwägungen daher entscheidend.

Bzgl. der Verlässlichkeit bedeutet das konkret, dass das System insbesondere Bestellungen persistent halten sollte, damit für den Kunden keine Probleme beim Bestellvorgang auftreten. Außerdem sollte die Benutzbarkeit des Systems besonders intuitiv sein, damit der Nutzer sich einfach durch die Anwendung navigieren kann. Hier spielt die GUI eine zentrale Rolle.

Es sollten bei Nutzereingaben direkt Fehler bzw. mögliche Komplikationen geprüft und in dem Moment ggf. eine Fehlermeldung angezeigt werden. Fehler bei Aufgabe und Bearbeitung von Bestellungen sollten abgefangen und der Vorgang abgebrochen werden. Ansonsten gibt es keine kritischen Fehler, sodass das System fehlertolerant implementiert werden kann, wobei lediglich Fehlerbeschreibungen gespeichert und dem Geschäftsführer angezeigt werden können sollten.

Damit aber möglichst wenige Fehler auftreten und eine gute Benutzererfahrung aufrechterhalten werden kann, sollte das System robust sein. Die Robustheit muss daher insbesondere durch hohe Test-Coverage der Logik überprüft werden.

Das Programm soll zwar nur auf einem Lokalen System laufen, nichtsdestotrotz ist der Schutz von Daten zu beachten, da verschiedene Personen auf das System zugreifen können. Dies wird umgesetzt, indem jeder Akteur seinen eigenen Zugang zum System hat und auch nur die Daten einsehen kann, die er einsehen muss. Der persönliche Zugang wird mit Hilfe eines Passworts geschützt, welches gehasht wird. Programmintern wird darauf geachtet, dass in Packages unterteilt wird, damit Klassen nur auf Klassen zugreifen, auf die sie zugreifen müssen. Insbesondere mit Hinblick auf zukünftige Erweiterungen und Expansionen des Systems ist die Sicherheit nicht zu vernachlässigen.

Das Programm sollte darüber hinaus auch gut erweiterbar sein, da es ggf. bei einer Skalierung des Unternehmens viele weitere Funktionen gibt, die noch später implementiert werden können. Das Programm legt dabei ein Fundament für zukünftige Modifizierungen und Anpassungen bereit, sodass dies, entsprechend den Anforderungen des Kunden, einfach gewartet und erweitert werden kann. Insbesondere das Layout und Design sollten gut modifizierbar sein, um einen langfristigen Nutzen zu gewährleisten.

Eine gute Lesbarkeit des Codes ist für die Erweiterbarkeit und Entwicklung des Programms im Team essenziell. Dazu tragen ausführliche Kommentare im Code und die Dokumentation mit Javadoc bei. Dies soll für eine gute Zusammenarbeit für das aktuellen Team, aber auch für spätere Entwicklungsteam sorgen.

Eine gewisse Geschwindigkeit ist wegen der rein lokal laufenden Software bereits gegeben. Zudem wirkt sich die Serialisierung nicht stark auf die Geschwindigkeit des Systems aus, da die Serialisierung nur beim Logout oder schließen des Systems die internen Listen speichert. Die serialisierten Daten werden einmalig beim Erzeugen der jeweiligen Verwalter-Klassen geladen und dann intern bearbeitet, sodass auch das Laden der Dateien keine großen Auswirkungen auf die Laufzeiten während der Nutzung des Systems darstellt. Da das System auch nur lokal arbeitet, reicht das einmalige Laden und Speichern aus, weil die Listen nicht synchronisiert werden müssen.

Das Programm kommt mit kleinen Daten aus, weshalb Speicherbedarf und Datendurchsatz keine große Rolle spielen. Bestellungen kopieren relevante Daten wie Name und Preis der Produkte, damit Kunden auch nach Entfernung oder Bearbeitung der Produkte die relevanten Informationen der Bestellung einsehen können. Ansonsten kommt das Programm ohne Duplikate aus. Des Weiteren werden Bilder, Design und Icons mit dem Programm übergeben. Dies stellt sicher, dass diese Medien immer zu Verfügung stehen. Dadurch bedarf es keiner dauerhaften Interverbindung für das Laden dieser Medien.

Zusammenfassend kann man sagen, dass die Speicherung in drei Kategorien eingeteilt werden sollte. Einmal die wichtige und direkte Speicherung von Daten, wie zum Beispiel bei Registrierungen, Bestellungen und Sortiment Verwaltung. Außerdem die Speicherung von Information, welche sich ändern könnten, aber man den Informationsstand vom jeweiligen Zeitpunkt haben möchte, wie von Bestellung oder Umsätze. Als letztes gibt es noch die Datenpacketen, welche man nur nach einer Abmeldung speichern muss. Darunter würde der Warenkorb des Kunden fallen.

Für die Berechnung der anfallenden Kosten der zu entwickelnden Software sind verschiedene Kostenfaktoren zu kalkulieren. Hierzu sind die Kosten für Personal, Materialien/Arbeitsgerät, Räumlichkeiten und ggf. Lizenzkosten von genutzter Software, Bilder oder Designs von Drittanbietern zu berücksichtigen. Zudem ist ebenfalls zu beachten, dass ein kurzer Zeitrahmen, der für die Entwicklung der Software zur Verfügung steht, die Arbeitsintensivität der Mitarbeiter deutlich erhöht und somit auch in der Vergütung der Arbeitsstunden zu Buche schlägt und berechnet werden muss. Dies sorgt ebenfalls für eventuellen Einschränkungen von Funktionen in der ersten Version, welche mit weiteren späteren Updates hinzugefügt werden müssen.

Um den Aufwand und die Kosten genauer zu bestimmen ist eine Kalkulierung mit der Function-Point-Methode sinnvoll. Insgesamt beläuft sich der berechnete Aufwand auf: 2.3 MM_{IBM} + (86 FP - 50 FP) / (100FP - 50FP) * (5.6 MM_{IBM} - 2.3MM_{IBM}) = **4.676MM_{IBM}.** Dieser Wert erfolgt aus der Umrechnung der Function-Points mittels Interpolation. Als Referenz-Mitarbeitermonate wurden hier Zahlen von IBM verwendet (Details siehe Anhang). Umgerechnet auf unser Entwicklerteam aus Informatikstudierenden im Gegensatz zu erfahrenen Softwareentwicklern von IBM sind dies ungefähr 10 Mitarbeitermonate, was der Länge und Gruppengröße des Softwarepraktikums entspricht.

Die Datenoperationen, die die Basis der Function-Point-Methode setzen sich aus den folgenden Funktionen (mit zugehörigeren Komplexitäten) zusammen:

External Inputs

Produkt erstellen	(einfach)	Preis, Beschreibung, Name, Bild angeben
Produkt bearbeiten	(einfach)	Preis, Beschreibung, Name, Bild überarbeiten
Produkt löschen	(einfach)	Ein Produkt mit geringer Attributanzahl löschen
K_Bestellung erstellen	(einfach)	Zahlungsmethode & Datum angeben
K_Bestellung bearbeiten	(einfach)	Zahlungsmethode & Datum anpassen
Profil bearbeiten	(einfach)	Vorname, Nachname & Adresse anpassen

GF: Nutzer hinzufügen	(einfach)	Nutzername & Passwort
K: Registrieren	(einfach)	Nutzer haben geringer Attribut-Input-Anzahl
E_Bestellung erstellen	(einfach)	Bestellung
External Outputs		
Statistik	(mittel)	Ein Datenbestand mit >20 Elementen
FehlerLogs	(einfach)	Ein Datenbestand mit geringer Fehleranzahl
Bestseller	(mittel)	Ein Datenbestand mit >20 Elementen
External Inquiries		
Filterlogik der Produktsuche	(mittel)	Ein Datenbestand mit >20 Produkten
Filterlogik der K_Bestellungen	(mittel)	Ein Datenbestand mit >20 Produkten
Filterlogik der E_Bestellungen	(mittel)	Ein Datenbestand mit >20 Produkten
Internal Logical Files		
Serialisierung Produkte	(mittel)	Datenbestand mit >50 Produkten
Serialisierung Kunden	(mittel)	Datenbestand mit >50 Kunden
Serialisierung E_Bestellungen	(einfach)	Datenbestand mit >50 Bestellungen
Serialisierung Kategorien	(einfach)	Datenbestand mit <50 Kategorien
Serialisierung Fehler	(einfach)	Datenbestand mit <50 Fehlern
External Interface Files		
-		

Die Nützlichkeit des Systems für Benutzer ist klar erkennbar; die Arbeit eines Unternehmens, welches Lebensmittel verkauft, wird durch die entwickelte Software deutlich vereinfacht. Nicht nur, dass Kunden der Bestellvorgang durch eine digitale Benutzeroberfläche vereinfacht wird, auch die Arbeit der Mitarbeitenden wird vereinfacht. Lieferanten können durch eine Bestellübersicht und eine Bestelldetailansicht effizienter arbeiten und Einkäufer können durch die farbliche Hervorhebung direkt erkennen, welche Produkte nachbestellt werden müssen. Auch für die Entwicklung des Unternehmens aus Betriebswirtschaftlicher Sicht ist das Programm nützlich: dem Geschäftsführer werden Statistiken, Bestsellerlisten und Umsätze angezeigt, welche dabei helfen zu erkennen, welche Produkte rentabel sind und wie sich das Sortiment verändern sollte.

Eine gute Nutzbarkeit des Systems durch den Kunden ist durch eine intuitive Benutzeroberfläche gegeben. Außerdem setzt das Design darauf, dass Kunden bereits Erfahrung mit ähnlichen Bestellsystemen gemacht haben. Die Nutzbarkeit durch Mitarbeitende des Unternehmens ist durch ein Nutzerhandbuch gegeben. Diese Nutzbarkeit werden durch klare Icons geregelt, welche für eine übersichtlichere Oberfläche sorgt als textuelle Beschreibungen. Zudem besitzen Icons Tooltips, welche dem Nutzer nochmals unterstützen.

2.2 Systemzerlegung

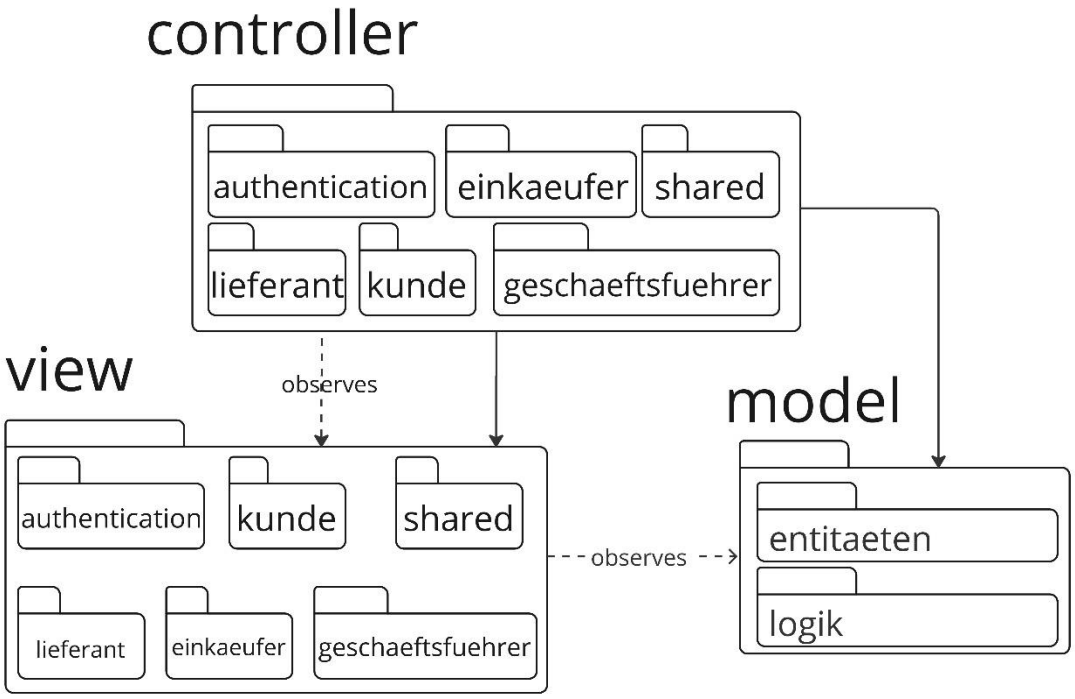
Für die Darstellung der Packages nutzen wir ein Model-View-Controller-Modell. Dieses greift unsere Überlegungen eines großen Steuerungspackages durch ein Controllerpackage, welches Zugriffsrechte auf das View- und Modelpackage hat, auf.

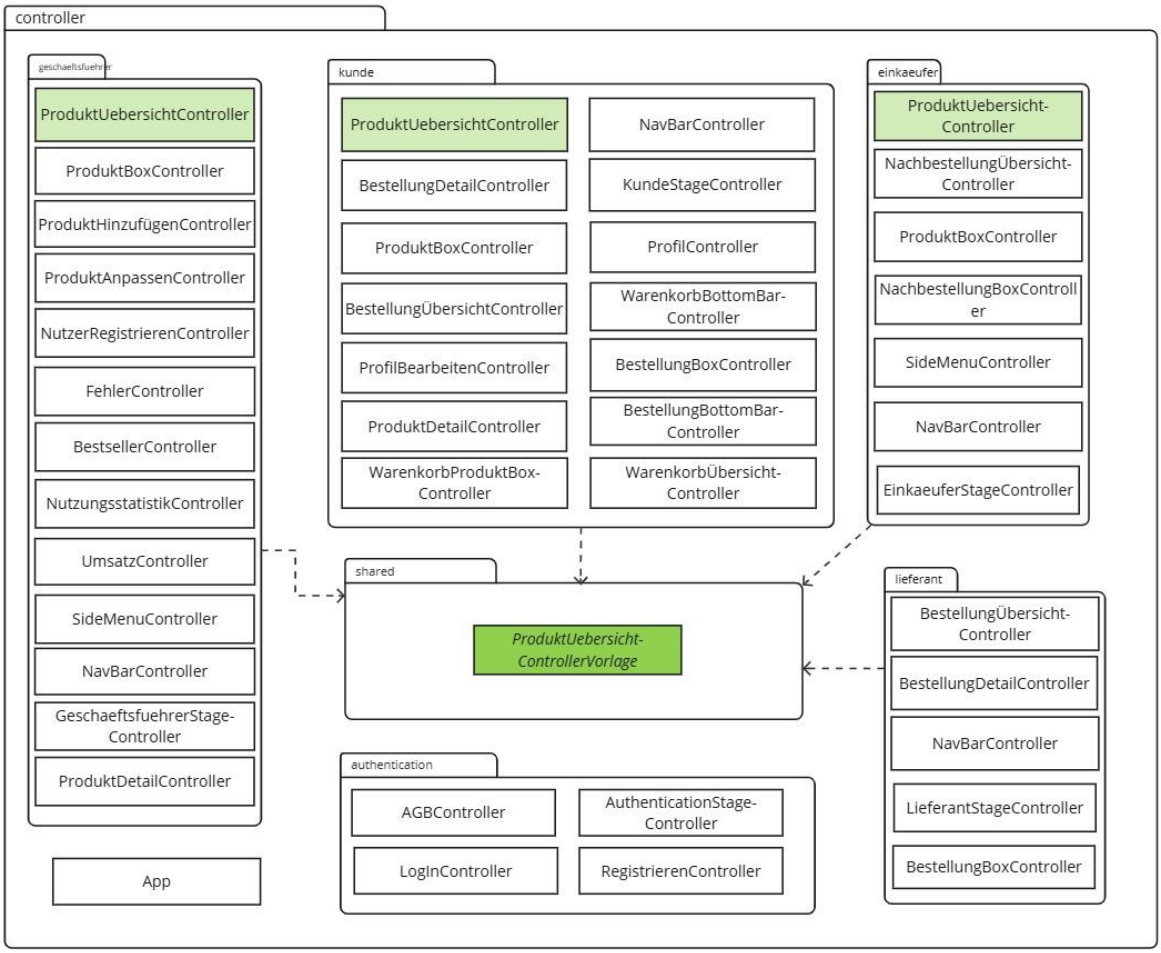
Das Modelpackage enthält alle Entitätsklassen in einem Subpackage, sowie auch ein Subpackage aller Klassen, die zur Datenerhaltung und Datenverwaltung mit Daten arbeiten, jedoch keinen Einfluss auf die View des Systemnutzers haben.

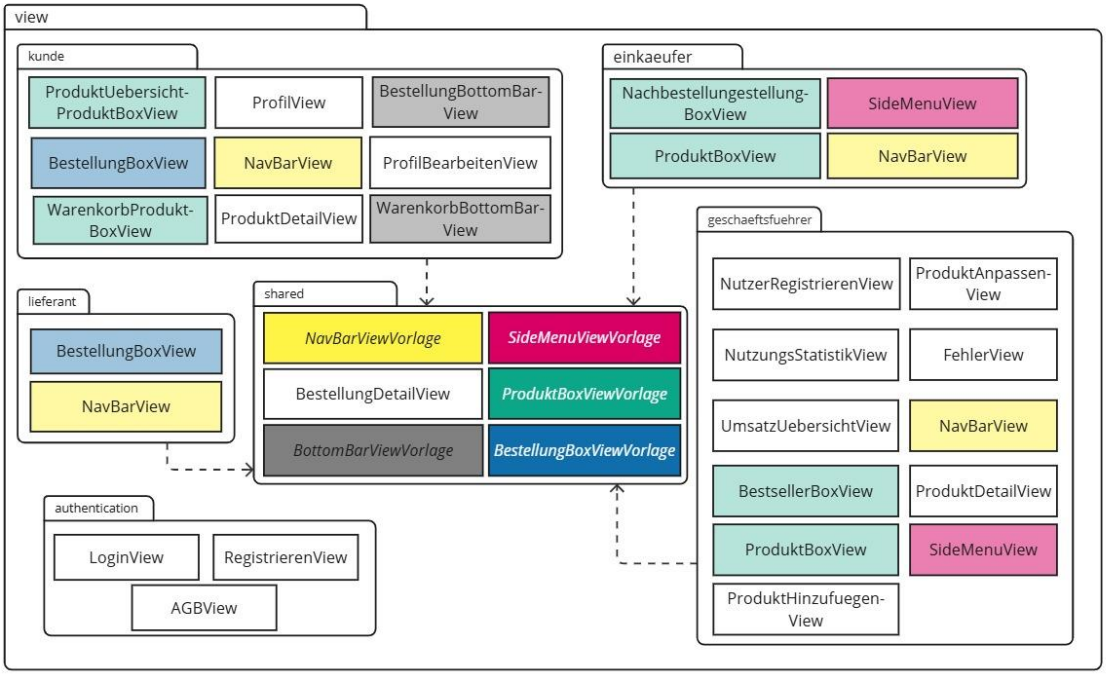
Das Controllerpackage beinhaltet alle Datenverarbeitungsklassen, die Einfluss auf die Nutzeroberfläche haben. Die Klassen sind nach Systemakteuren in Packages unterteilt, um Klassenbenennungen und die Bezüge zu den Darstellungsklassen klarer zu machen. Um Codeduplikationen zu vermeiden gibt es ein shared-Package welches Oberklassen für Vererbungsbeziehungen im Controllerpackage bereitstellt. Das Authenticationpackage stellt Funktionen zur Registrierung und Login bereit und übergibt dann die Kontrolle an einen Stagecontroller je nach Akteur.

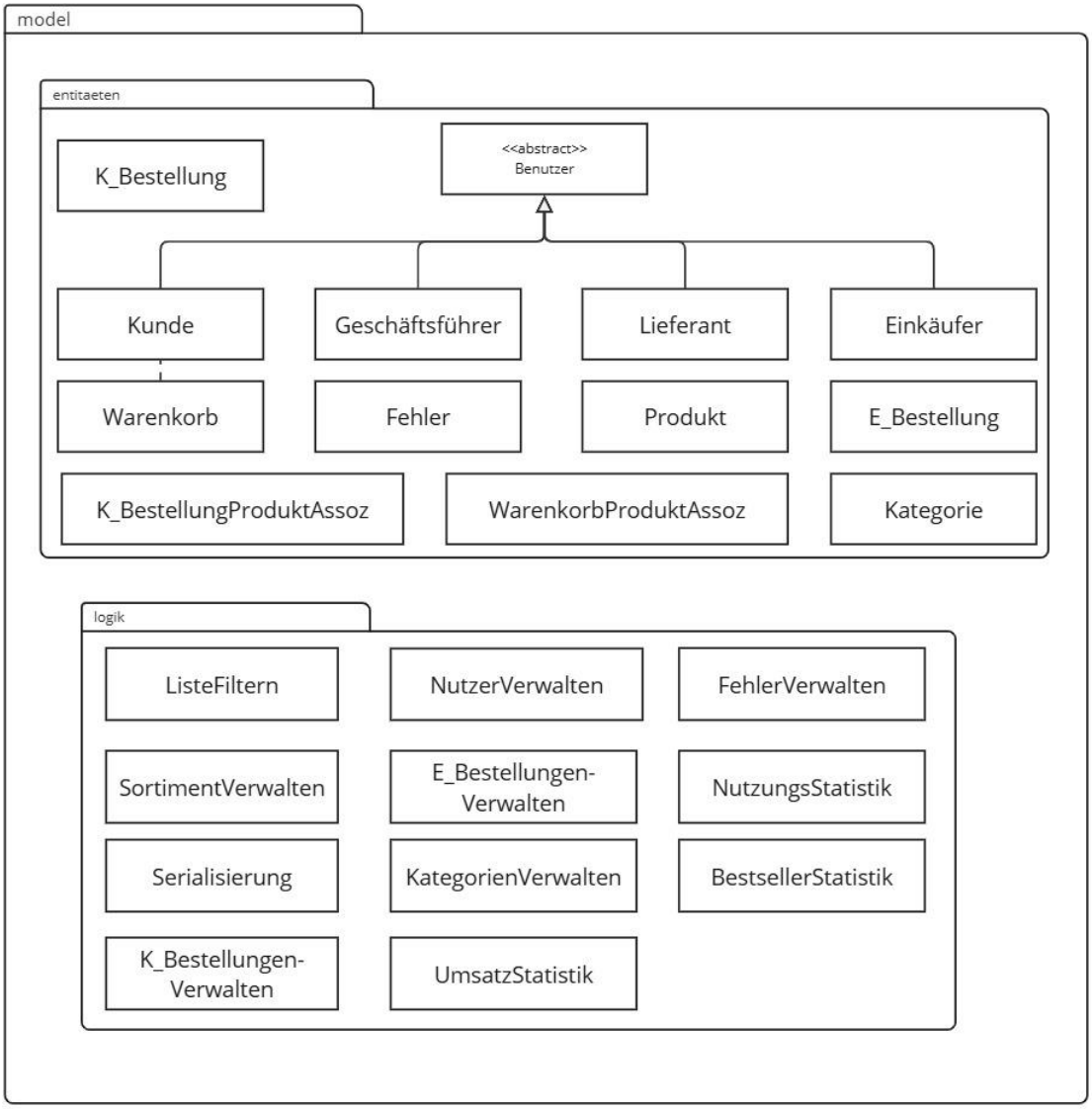
Das Viewpackage enthält alle Klassen, die gegebene Daten auf die Oberfläche projizieren. Diese sind in Packages nach Akteur eingeteilt, welche Assoziationen zum Package „Shared Interfaces“ besitzen. Es bildet ein Grundgerüst, auf dem die anderen Views aufbauen können und enthält alle Kerndaten von wiederverwendbaren Views.

Zur besseren Übersichtlichkeit wurde eine farbliche Kodierung zur Kennzeichnung von Vererbungsbeziehungen genutzt. Die Oberklasse hat eine stärkere Farbsättigung und Unterklassen sind in der gleichen Farbe mit geringerer Sättigung eingefärbt. So erbt beispielsweise NavBarView von NavBarViewVorlage.




controller
geschaeftsfuehrer
ProduktUebersichtController
ProduktBoxController
ProduktHinzufügenController
ProduktAnpassenController
NutzerRegistrierenController
FehlerController
BestsellerController
NutzungsstatistikController
UmsatzController
SideMenuController
NavBarController
GeschaeftsfuehrerStage-Controller
ProduktDetailController
App
kunde
ProduktUebersichtController
NavBarController
BestellungDetailController
KundeStageController
ProduktBoxController
ProfilController
BestellungÜbersichtController
WarenkorbBottomBar-Controller
ProfilBearbeitenController
BestellungBoxController
ProduktDetailController
BestellungBottomBar-Controller
WarenkorbProduktBox-Controller
WarenkorbÜbersicht-Controller
einkaeufer
ProduktUebersicht-Controller
NachbestellungÜbersicht-Controller
ProduktBoxController
NachbestellungBoxController
SideMenuController
NavBarController
EinkaeuferStageController
shared
ProduktUebersicht-ControllerVorlage
lieferant
BestellungÜbersicht-Controller
BestellungDetailController
NavBarController
LieferantStageController
BestellungBoxController
authentication
AGBController
AuthenticationStage-Controller
LogInController
RegistrierenController


view
kunde
ProduktUebersicht-ProduktBoxView
ProfilView
BestellungBottomBar-View
BestellungBoxView
NavBarView
ProfilBearbeitenView
WarenkorbProdukt-BoxView
ProduktDetailView
WarenkorbBottomBar-View
einkaeufer
Nachbestellungestellung-BoxView
SideMenuView
ProduktBoxView
NavBarView
geschaeftsfuehrer
NutzerRegistrierenView
ProduktAnpassen-View
NutzungsStatistikView
FehlerView
UmsatzUebersichtView
NavBarView
BestsellerBoxView
ProduktDetailView
ProduktBoxView
SideMenuView
ProduktHinzufuegen-View
lieferant
BestellungBoxView
NavBarView
shared
NavBarViewVorlage
SideMenuViewVorlage
BestellungDetailView
ProduktBoxViewVorlage
BottomBarViewVorlage
BestellungBoxViewVorlage
authentication
LoginView
RegistrierenView
AGBView


model
entitaeten
K_Bestellung
<<abstract>>
Benutzer
Kunde
Geschäftsführer
Lieferant
Einkäufer
Warenkorb
Fehler
Produkt
E_Bestellung
K_BestellungProduktAssoz
WarenkorbProduktAssoz
Kategorie
logik
ListeFiltern
NutzerVerwalten
FehlerVerwalten
SortimentVerwalten
E_Bestellungen-
Verwalten
NutzungsStatistik
Serialisierung
KategorienVerwalten
BestsellerStatistik
K_Bestellungen-
Verwalten
UmsatzStatistik

2.3 Verwendung existierender Software-Komponenten

Java.io:

Java.io ist ein in Java integriertes Package, welches Klassen und Interfaces bereitstellt, mit denen Klassen (de-)serialisiert und Daten auf dem Dateisystem gespeichert und gelesen werden können. Hier nutzen wir es, da wir diverse Informationen persistent speichern müssen, wie z.B. Kundendaten, Produkte oder Bestellungen. Zudem muss dieses Paket nicht erst installiert werden, sondern steht direkt zur Verfügung.

JavaFX:

JavaFX ist eine plattformübergreifende Bibliothek, mit der man einfach Desktop-Applikationen mit GUIs erstellen kann. Wir nutzen es in diesem Projekt, weil es viele Funktionen, wie z.B. Tabellen, Buttons oder Textfelder, zur Verfügung stellt. Zudem Nutzen wir es, um das GUI möglichst simpel erstellen zu können und plattformübergreifende Kompatibilität zu gewährleisten. Die Verwendung von JavaFX war außerdem vorgeschrieben.

Java.util:

Java.util ist ein weiteres in Java integriertes Paket und stellt viele hilfreiche Klassen und Interfaces bereit, welche genutzt werden können, um Daten zu halten und sie zu Verarbeiten. Wir nutzen es in diesem Projekt, da es Klassen und Interfaces wie Listen und Mengen bereitstellt, in denen wir diverse Informationen halten können, wie z.B. Produkte, Kategorien, etc. Zudem ist dies ein weiteres Paket, welches bereits in Java integriert ist, was die Verwendung einfacher macht, da es nicht erst installiert werden muss.

JUnit:

JUnit ist ein Testing-Framework, mit dem man Unit-Tests für Klassen schreiben kann. Wir nutzen es in diesem Projekt, damit die Klassen in diesem Projekt auf Fehler getestet werden können und um die korrekte Funktionalität zu überprüfen. Zudem nutzen wir es, da es gute Integrationen in vielen IDEs bietet.

Java.lang.Math:

Java.lang.Math ist eine in Java integrierte Klasse, welche einige Methoden und Konstanten bereitstellt zur Berechnung vieler Mathematischer Ausdrücke. In diesem Projekt verwenden wir es, da es bei der Berechnung der Statistiken helfen kann. Zudem ist dies auch in Java integriert, womit es nicht erst installiert werden muss.

Java.text:

Java.text ist ebenfalls ein Paket, welches in Java integriert ist. Es stellt Klassen und Interfaces bereit, mit denen man u.a. mit Text und Zahlen arbeiten kann. Es wird hier verwendet, wegen der Formatierung von Text. Damit können wir z.B. Preise von Produkten in das korrekte Format bringen. Zudem gilt, dass, wie alle in Java integrierten Pakete, dieses ebenfalls nicht erst installiert werden muss.

Passwort Hashing:

Dies ist eine Beispielklasse aus dem Learnweb, mit der Passwörter gehasht werden können. In diesem Projekt nutzen wir sie, weil die Passwörter der Nutzer nicht in Klartext, sondern gehasht

gespeichert werden soll. Außerdem nutzen wir diese Klasse, da wir sie dann nicht extra implementieren müssen, womit wir wertvolle Zeit sparen.

Java.time:

Java.time ist ein in Java integriertes Paket, welches viele Klassen beinhaltet, die das Arbeiten mit Zeit erleichtern. Es wird hier genutzt, da einige Funktionen des Programms mit Zeit arbeiten müssen, wie z.B. die Wunschlieferzeit der Bestellungen. Zudem erleichtert es uns die Arbeit stark, da mit Zeiten programmieren kompliziert ist, und uns so die Implementation unserer eigenen Zeitklassen erspart bleibt.

2.4 Management persistenter Daten

Im Bestellsystem müssen diverse Daten langfristig gespeichert werden, um sie beim Starten des Systems aufzurufen und darauf Einfluss nehmen zu können. Zur Speicherung der Daten wird die interne von Java gegebene Serialisierung konsequent genutzt. Insbesondere werden *Entitätsobjekte* serialisiert und somit konsistent gespeichert.

Die wichtigsten zu speichernden Daten sind die Nutzerdaten. Dabei werden Tupel von Nutzername und einem durch Hashing gesicherten Passwort in einer Liste gespeichert. Außerdem werden die verschiedenen Nutzertypen als *Nutzerobjekt* in verschiedenen Listen gespeichert, um sie differenzieren zu können, ohne dem Nutzerobjekt ein weiteres Attribut zu verleihen.

Das Produktsortiment wird langfristig gespeichert, indem zum einen eine Liste mit allen im System existierende *Produkte* gespeichert wird, sowie auch eine Liste an *Produktkategorien*, die für jede Kategorie alle Produkte in dieser Kategorie kennt. Weiter werden aktuelle *Warenkörbe* von Kunden als Tupel von Listen von Produkten und deren Anzahl gespeichert, um Warenkörbe auch nach dem Beenden des Systems zu sichern. Dazu passend werden alle von Kunden getätigte *Bestellungen* mit den gleichen Dateninhalten wie dem Warenkorb und zusätzlich einem Status sowie Eckdaten der Bestellung (Aufgabedatum, Bestellnummer, Wunschtermin, Zahlart) gesichert, um dem Kunden langfristigen Zugriff darauf zu ermöglichen, sowie auch für die Berechnung von Umsatzstatistiken für den Geschäftsführer.

Auf Statistiken bezogen ist auch eine Sicherung der im System auftretenden Fehler wichtig. Diese wird immer beim Abfangen eines Fehlers vorgenommen und in einer Liste von *Fehlern* gesichert, damit der Geschäftsführer jederzeit Zugriff auf alle im System aufgetretenen Fehler hat.

Zuletzt werden alle vom Einkäufer getätigten *Nachbestellungen* gespeichert. Diese Datenpakete beinhalten ein Tupel von Produkt und Anzahl sowie eine Bestellnummer und das Aufgabedatum.

3. Objektentwurf

3.1 Abwägung des Objektentwurfs

Um das Klassendiagramm mit den Entitätsklassen zu entwerfen, muss man sich zunächst Gedanken um Vererbungsbeziehungen machen. Es leuchtet ein, gemeinsame Attribute von allen Akteuren (nutzername, passwort) in eine gemeinsame abstrakte Oberklasse *Nutzer* auszulagern. Weitere Vererbungsbeziehungen sind nicht sinnvoll, da die restlichen Entitäten außer Assoziationen unabhängig voneinander sind. Insbesondere hat die Klasse *Fehler* keine Beziehungen zu anderen Entitäten

Des Weiteren ist es interessant sich Gedanken um Assoziationen zwischen Entitäten zu machen: während *Warenkorb* und *K_Bestellung* direkt in der Klasse *Kunde* gespeichert werden, hat beispielsweise der *Einkäufer* keine Assoziation zur Klasse *E_Bestellung*, weil die Nachbestellungen unabhängig vom Einkäufer gespeichert werden. Ein *Lieferant* hingegen ist direkt einer oder mehreren K_Bestellungen zugewiesen.

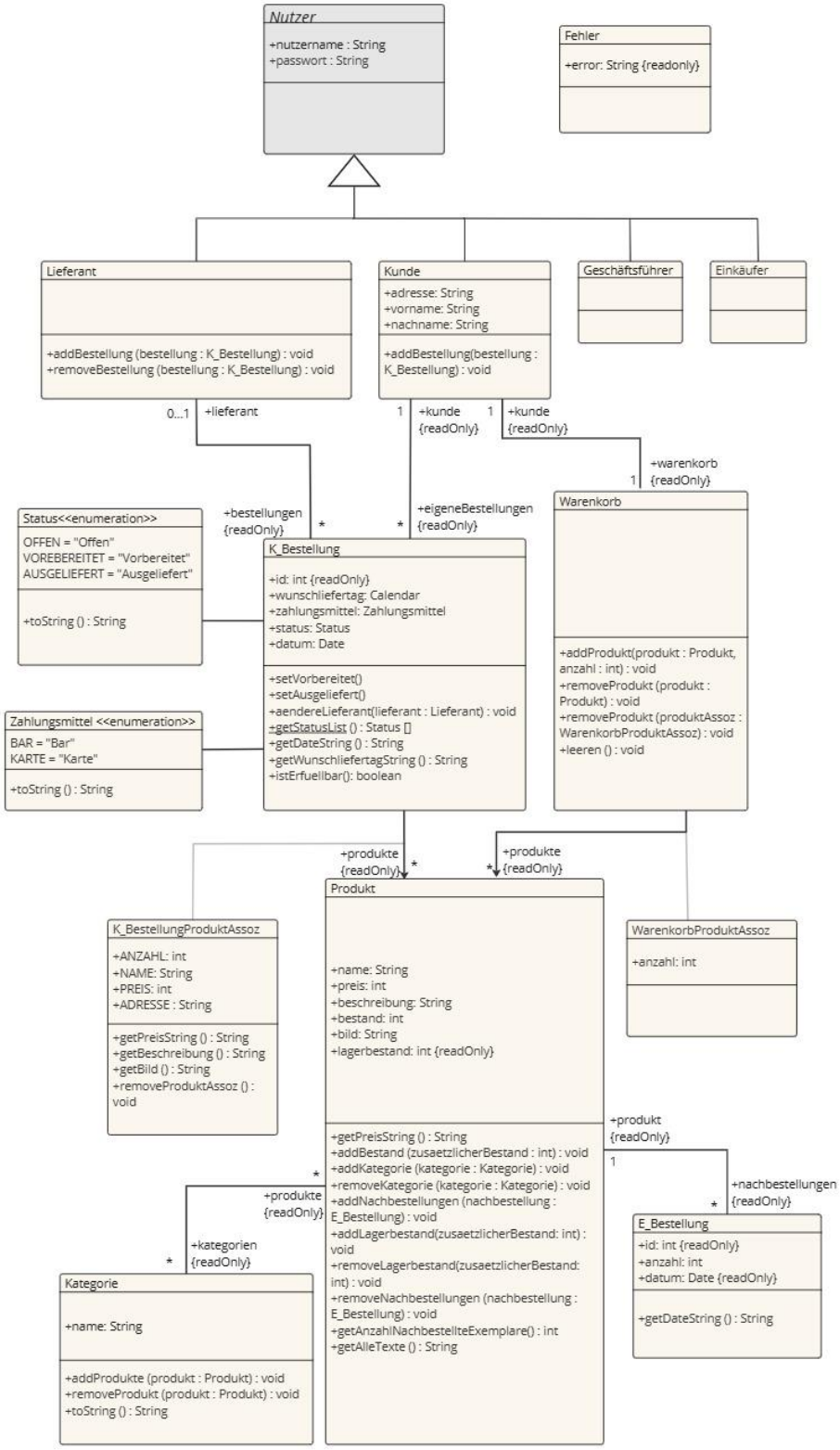
Obwohl der *Geschäftsführer* das Produktsortiment verwalten kann, hat er keine Assoziation zur Klasse *Produkt*, da die Liste der Produkte nicht in der Entitätsklasse, sondern in einer Logikklasse, beziehungsweise der Serialisierung gespeichert ist.

Die Entitätsklasse *Kategorie* ist nur für das Produkt relevant, da dieses eine Liste an Kategorien speichert unter die es fällt.

Besonders hervorzuheben sind die Assoziationsklassen, die für die Beziehung Warenkorb-Produkt und K_Bestellung-Produkt gebraucht werden um eine Liste an verschiedenen Produkten mit verschieder Anzahl zu speichern. Außerdem werden in der Assoziationsklasse *K_BestellungProduktAssoz* einige Attribute der Produkte gespeichert, um gegebenenfalls in vergangenen Bestellungen bereits gelöschte Produkte anzeigen zu können.

Die Zahlungsmethode und der Status einer K_Bestellung werden als Enum Zahlungsmethode (Bar oder Karte) und Status (Offen, Vorbereitet oder Ausgeliefert) gespeichert.

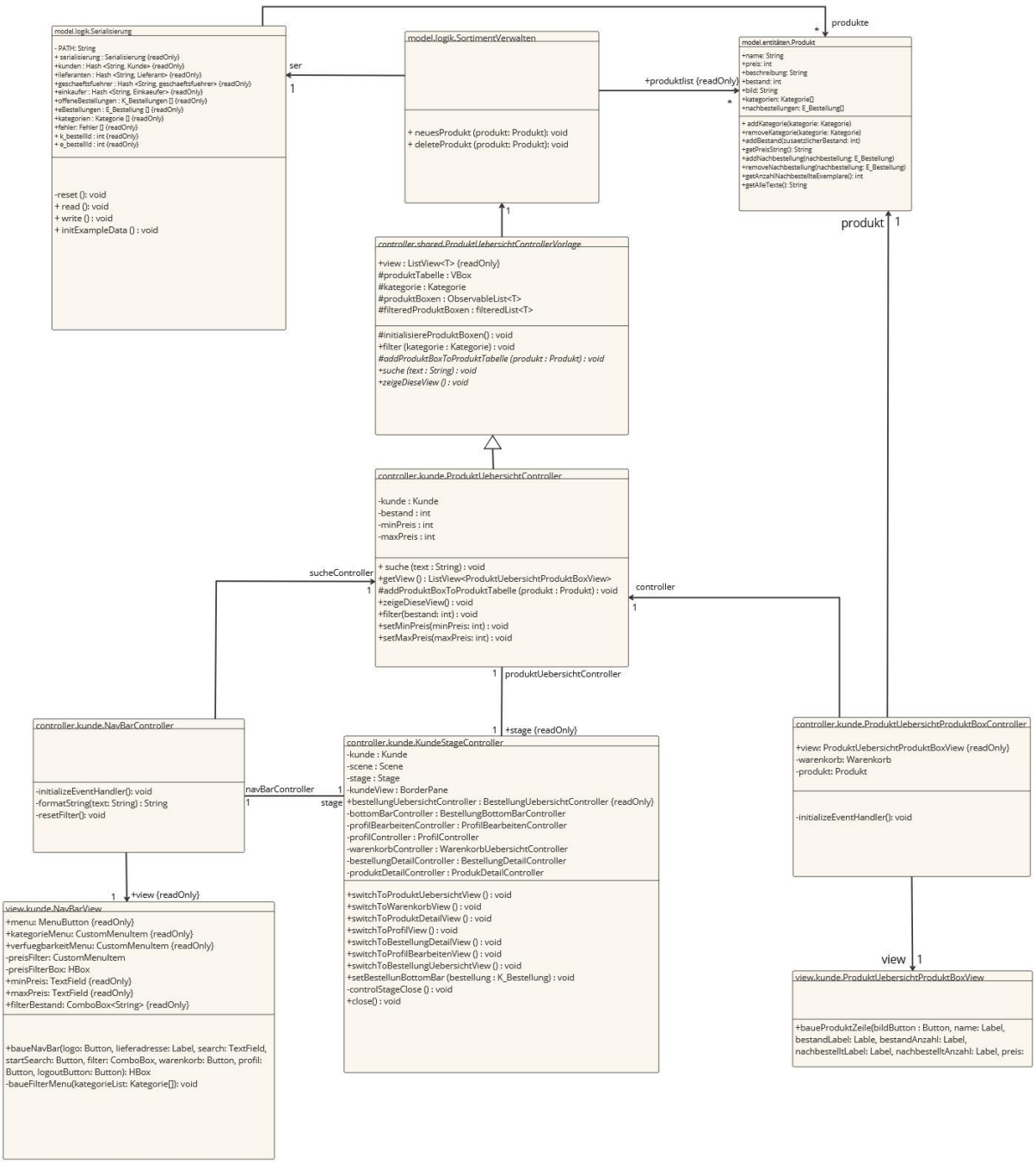
3.2 Klassenmodell der Entitäten



3.3 Dokumentation weiterer interessanter Ausschnitte des Entwurfsklassenmodells

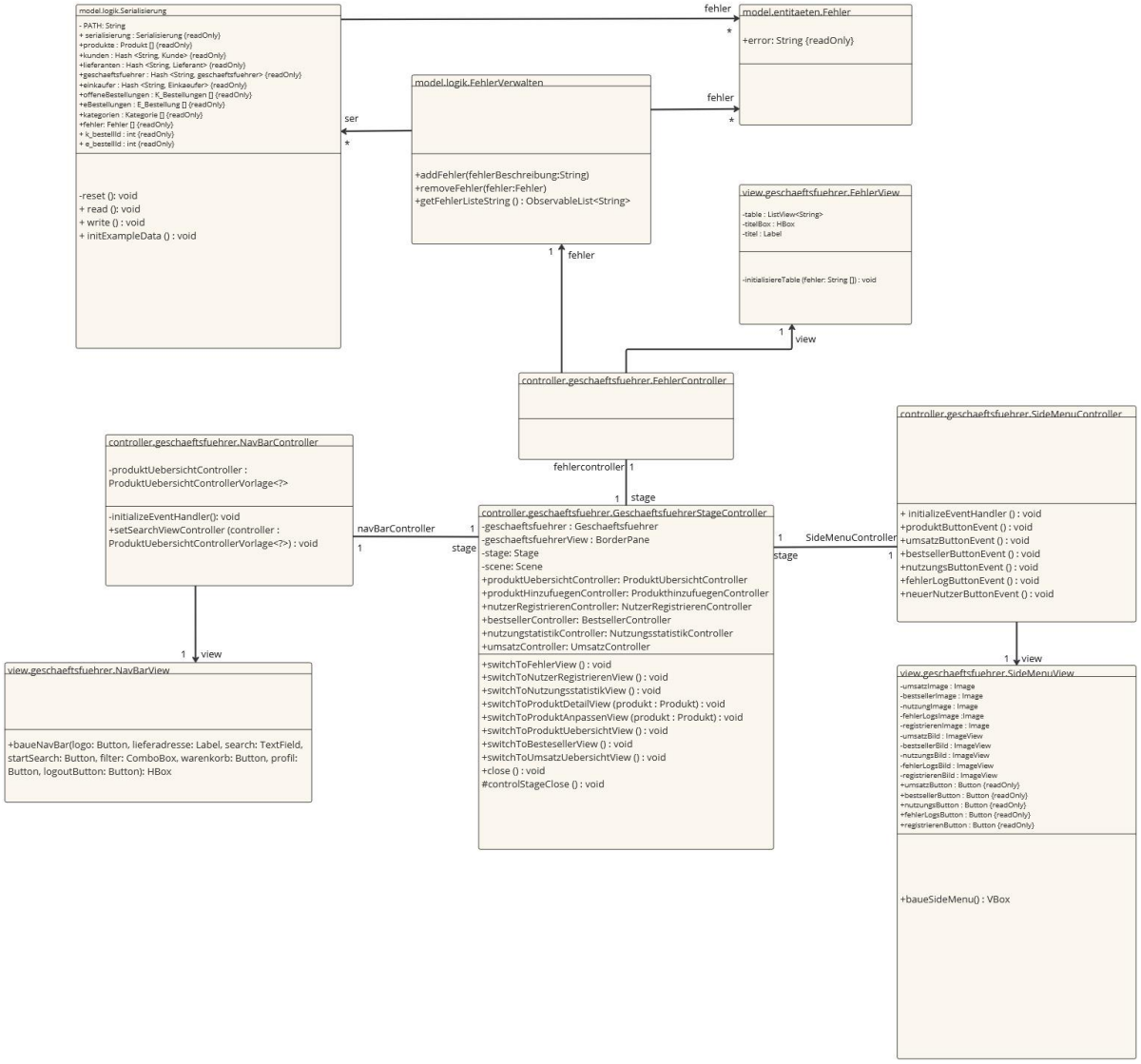
Produktübersicht von Kunde

Das folgende Klassendiagramm stellt die relevanten Klassen dar, die benötigt werden damit ein Kunde sich die Produktübersicht anzeigen lassen kann. Die Logik des Produktsortiments befindet sich im Model-Package und besteht aus den Klassen Serialisierung, Produkt und SortimentVerwalten. Diese Klassen stellen der Klasse ProduktUebersichtControllerVorlage im Controller-Package Zugriff auf die Produktdaten zur Verfügung. Der KundeStageController hat Zugriff auf die relevanten Controller, um die View-Klassen im View-Package anzulegen und die JavaFX Szene auf der aktuellen stage zusammenzusetzen. Diese besteht aus einer NavBarView und mehreren ProduktUebersichtProduktBoxViews, welche zu einer Produkttabelle zusammengefügt werden. Dabei erben ProduktUebersichtProduktBoxView und NavBarView jeweils von einer Oberklasse, welche im Package shared liegt. Der NavBarController hat dabei außerdem Zugriff auf den ProduktUebersichtController um die Suche nach Produkten in der Übersicht zu realisieren.


model.logik.Serialisierung
- PATH: String
+ serialisierung : Serialisierung {readOnly}
+kunden : Hash <String, Kunde> {readOnly}
+lieferanten : Hash <String, Lieferant> {readOnly}
+geschaeftsfuehrer : Hash <String, geschaeftsfuehrer> {readOnly}
+einkaufer : Hash <String, Einkaeufer> {readOnly}
+offeneBestellungen : K_Bestellungen [] {readOnly}
+eBestellungen : E_Bestellung [] {readOnly}
+kategorien : Kategorie [] {readOnly}
+fehler: Fehler [] {readOnly}
+ k_bestellId : int {readOnly}
+ e_bestellId : int {readOnly}
-reset (): void
+ read (): void
+ write () : void
+ initExampleData () : void
ser
1
model.logik.SortimentVerwalten
+ neuesProdukt (produkt: Produkt): void
+ deleteProdukt (produkt: Produkt): void
+produktlist {readOnly}
*
*
produkte
model.entitäten.Produkt
+name: String
+preis: int
+beschreibung: String
+bestand: int
+bild: String
+kategorien: Kategorie[]
+nachbestellungen: E_Bestellung[]
+ addKategorie(kategorie: Kategorie)
+removeKategorie(kategorie: Kategorie)
+addBestand(zusaetzlicherBestand: int)
+getPreisString(): String
+addNachbestellung(nachbestellung: E_Bestellung)
+removeNachbestellung(nachbestellung: E_Bestellung)
+getAnzahlNachbestelltteExemplare(): int
+getAlleTexte(): String
produkt
1
1
controller.shared.ProduktUebersichtControllerVorlage
+view : ListView<T> {readOnly}
#produktTabelle : VBox
#kategorie : Kategorie
#produktBoxen : ObservableList<T>
#filteredProduktBoxen : filteredList<T>
#initialisiereProduktBoxen() : void
+filter (kategorie : Kategorie) : void
#addProduktBoxToProduktTabelle (produkt : Produkt) : void
+suche (text : String) : void
+zeigeDieseView () : void
controller.kunde.ProduktUebersichtController
-kunde : Kunde
-bestand : int
-minPreis : int
-maxPreis : int
+ suche (text : String) : void
+getView () : ListView<ProduktUebersichtProduktBoxView>
#addProduktBoxToProduktTabelle (produkt : Produkt) : void
+zeigeDieseView() : void
+filter(bestand: int) : void
+setMinPreis(minPreis: int) : void
+setMaxPreis(maxPreis: int) : void
sucheController
1
controller
1
1
produktUebersichtController
1
+stage {readOnly}
controller.kunde.NavBarController
-initializeEventHandler(): void
-formatString(text: String) : String
-resetFilter(): void
navBarController
1
1
stage
controller.kunde.KundeStageController
-kunde : Kunde
-scene : Scene
-stage : Stage
-kundeView : BorderPane
+bestellungUebersichtController : BestellungUebersichtController {readOnly}
-bottomBarController : BestellungBottomBarController
-profilBearbeitenController : ProfilBearbeitenController
-profilController : ProfilController
-warenkorbController : WarenkorbUebersichtController
-bestellungDetailController : BestellungDetailController
-produktDetailController : ProdukDetailController
+switchToProduktUebersichtView () : void
+switchToWarenkorbView () : void
+switchToProduktDetailView () : void
+switchToProfilView () : void
+switchToBestellungDetailView () : void
+switchToProfilBearbeitenView () : void
+switchToBestellungUebersichtView () : void
+setBestellunBottomBar (bestellung : K_Bestellung) : void
-controlStageClose () : void
+close() : void
controller.kunde.ProduktUebersichtProduktBoxController
+view: ProduktUebersichtProduktBoxView {readOnly}
-warenkorb: Warenkorb
-produkt: Produkt
-initializeEventHandler(): void
1
+view {readOnly}
view.kunde.NavBarView
+menu: MenuButton {readOnly}
+kategorieMenu: CustomMenuItem {readOnly}
+verfuegbarkeitMenu: CustomMenuItem {readOnly}
-preisFilter: CustomMenuItem
-preisFilterBox: HBox
+minPreis: TextField {readOnly}
+maxPreis: TextField {readOnly}
+filterBestand: ComboBox<String> {readOnly}
+baueNavBar(logo: Button, lieferadresse: Label, search: TextField, startSearch: Button, filter: ComboBox, warenkorb: Button, profil: Button, logoutButton: Button): HBox
-baueFilterMenu(kategorieList: Kategorie[]): void
view
1
view.kunde.ProduktUebersichtProduktBoxView
+baueProduktZeile(bildButton : Button, name: Label, bestandLabel: Lable, bestandAnzahl: Label, nachbestelltLabel: Label, nachbestelltAnzahl: Label, preis:

Fehlerlogs von Geschäftsführer

Das folgende Klassendiagramm stellt die relevanten Klassen dar, die benötigt werden, damit sich ein Geschäftsführer die Fehlerlogs anzeigen lassen kann. Dazu werden die Entitätsklasse Fehler und die Logikklassen Serialisierung und FehlerVerwalten aus dem model-Package gebraucht. Diese bieten dem FehlerController Zugriff auf die Serialisierten Fehlerdaten. Die Klassen NavBarView, FehlerView und SideMenuView aus dem View-Package bilden die graphische Oberfläche für das Menü des Geschäftsführers, wobei SideMenu und NavBar wieder Unterklassen von Klassen aus dem shared Package sind. Der GeschaeftsfuehrerStageController kennt die relevanten Controller der View-Klassen, um seine stage mit der richtigen JavaFX Szene auszustatten.



4. Anhang

Details zur Funktion-Point-Analyse

FP	MM_{IBM}	MM_{VW}
50	2.3	-
100	5.6	-
150	9.5	-
200	13.9	11.7
250	18.6	19.3
300	23.6	27.1
350	28.9	35.0
400	34.4	43.0
450	40.1	51.1
500	46.1	59.6
550	52.2	68.2
600	58.5	77.0
650	65.0	86.1
700	71.6	95.3
750	78.4	104.8
800	85.3	114.6
850	92.4	124.7
900	99.6	135.2
950	106.9	146.0
1000	114.4	157.3
1100	129.6	181.3
1200	145.2	207.8
1300	161.3	237.8
1400	177.7	273.8
1500	194.6	319.1

Function Point Calculator

The Madison Utilities, Department of Computer Science, James Madison University

Total	Factor	FP
94	0.92	86

Direct Measure	Count Simple	Count Average	Count Complex	Weighted Measure
External Inputs (EIs)	9	0	0	27
External Outputs (EOs)	1	2	0	14
External Inquiries (EQs)	0	3	0	12
Internal Logical Files (ILFs)	3	2	0	41
External Interface Files (EIFs)	0	0	0	0

Clear

Value Adjustment Factor	0	1	2	3	4	5
The system requires reliable backup and recovery.						●
Specialized data communications are required.			●			
There are distributed processing functions.		●				
Performance is critical.		●				
The system runs in an existing, heavily utilized operational environment.	●					
The system requires on-line data entry.				●		
The on-line data entry requires transactions over multiple screens/operations.		●				
ILFs are updated on-line.	●					
The inputs, outputs, files or inquiries are complex.					●	
The internal processing is complex.						●
The code is designed to be reusable.				●		
Conversions /installation are included in the design.	●					
The system is designed for multiple installations in different organizations.	●					
The system is designed to facilitate change and ease of use.			●			